DS 320 Algorithms for Data Science	**Lecture #18**
Spring 2026	Prof. Kira Goldner and Jeffrey Considine

Linear Programming I

Why Linear Programming rocks:

- Incredibly general: Every problem we've seen so far can be formulated as a linear program.
- Computational tractable
  - In theory: Can be solved in polynomial time
  - In practice: Fast with input sizes up into the millions!
- Contains many properties that can be turned into useful algorithmic paradigms and analysis:
  - Duality:
    - Solve an easier equivalent problem.
    - How do we know when we're done?
  - Complementary Slackness and Strong Duality: something is optimal!

How to Think About Linear Programming

Comparison to Systems of Linear Equations

Think back to linear systems of equations. Such a system consists of m linear equations in real-valued variables $x_1, \ldots, x_n$:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{12}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{22}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{m2}x_n &= b_m. \end{aligned}$$

The a_{ij}'s and the b_i's are given; the goal is to check whether or not there are values for the x_j's such that all m constraints are satisfied. We used Gaussian elimination; "solved" meant that the algorithm returns a feasible solution, or correctly reports that no feasible solution exists.

What about inequalities? The point of linear programming is to solve systems of linear equations *and inequalities*. Moreover, when there are multiple feasible solutions, we would like to compute the "best" one.

Ingredients of a Linear Program

There is a convenient and flexible language for specifying linear programs, and we'll get lots of practice using it during this lecture. Sometimes it's easy to translate a computational problem into this language, sometimes it takes some tricks (we'll see examples of both).

To specify a linear program, you need to declare what's allowed and what you want.

Ingredients of a Linear Program

a. *Decision variables* $x_1, \ldots, x_n \in \mathbb{R}$.

b. *Linear constraints*, each of the form

$$\sum_{j=1}^{n} a_j x_j \quad (*) \quad b_i,$$

where $(*)$ could be $\leq, \geq$, or $=$.

c. A *linear objective function* of the form

$$\max \sum_{j=1}^{n} c_j x_j$$

or

$$\min \sum_{j=1}^{n} c_j x_j.$$

Comments:

- The a_{ij}'s, b_i's, and c_j's are *constants*, part of the input, hard-wired into the linear program (like 5, -1, 10, etc.).
- The x_j's are free, and it is the job of a linear programming algorithm to figure out the best values for them.
- When specifying constraints, there is no need to make use of both "$\leq$" and "$\geq$"inequalities—one can be transformed into the other just by multiplying all the coefficients by -1 (the a_{ij}'s and b_i's are allowed to be positive or negative).
- Equality constraints are superfluous, in that the constraint that a quantity equals b_i is equivalent to the pair of inequality constraints stating that the quantity is both at least bi and at most bi.
- There is also no difference between the "min" and "max" cases for the objective function—one is easily converted into the other just by multiplying all the c_j's by -1 (the c_j's are allowed to be positive or negative).

What's not allowed in a linear program? Terms like $x_j^2, x_j x_k, \log(1 + x_j)$, etc. So whenever a decision variable appears in an expression, it is alone, possibly multiplied by a constant (and then summed with other such terms). While these linearity requirements may seem restrictive, we'll see that many real-world problems can be formulated as or well approximated by linear programs.

A Simple Example

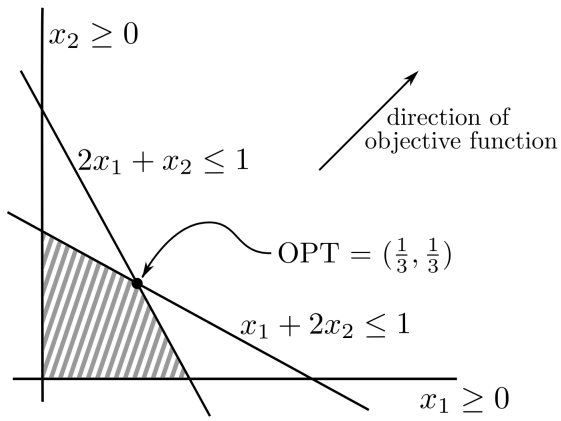

Figure 1: A toy example of a linear program.

To make linear programs more concrete and develop your geometric intuition about them, let's look at a toy example. (Many "real" examples of linear programs are coming shortly.) Suppose there are two decision variables x_1 and x_2—so we can visualize solutions as points (x_1, x_2) in the plane. See Figure 1. Let's consider the (linear) objective function of maximizing the sum of the decision variables:

$$\max x_1 + x_2.$$

We'll look at four (linear) constraints:

$$\begin{aligned} x_1 &\geq 0 \\ x_2 &\geq 0 \\ 2x_1 + x_2 &\leq 1 \\ x_1 + 2x_2 &\leq 1. \end{aligned}$$

The first two inequalities restrict feasible solutions to the non-negative quadrant of the plane. The second two inequalities further restrict feasible solutions to lie in the shaded region depicted in Figure . Geometrically, the objective function asks for the feasible point furthest in the direction of the coefficient vector $(1, 1)$—the "most northeastern" feasible point. Eyeballing the feasible region, this point is $(\frac{1}{3}, \frac{1}{3})$, for an optimal objective function value of $\frac{2}{3}$. This is the "last point of intersection" between a level set of the objective function and the feasible region (as one sweeps from southwest to northeast).

Geometric Intuition

While it's always dangerous to extrapolate from two or three dimensions to an arbitrary number, the geometric intuition above remains valid for general linear programs, with an arbitrary number of dimensions (i.e., decision variables) and constraints. Even though we can't draw pictures when there are many dimensions, the relevant algebra carries over without any difficulties. Specifically:

a. A linear constraint in n dimensions corresponds to a halfspace in $\mathbb{R}^n$. Thus a feasible region is an intersection of halfspaces, the higher-dimensional analog of a polygon.[1]

b. When there is a unique optimal solution, it is a vertex (i.e., "corner") of the feasible region.

A few edge cases:

a. There might be no feasible solutions at all. For example, if we add the constraint $x_1 + x_2 \geq 1$ to our toy example, then there are no longer any feasible solutions. Linear programming algorithms correctly detect when this case occurs.

b. The optimal objective function value is unbounded ($+\infty$ for a maximization problem, $-\infty$ for a minimization problem). Note a necessary but not sufficient condition for this case is that the feasible region is unbounded. For example, if we dropped the constraints $2x_1 + x_2 \leq 1$ and $x_1 + 2x_2 \leq 1$ from our toy example, then it would have unbounded objective function value. Again, linear programming algorithms correctly detect when this case occurs.

c. The optimal solution need not be unique, as a "side" of the feasible region might be parallel to the levels sets of the objective function. Whenever the feasible region is bounded, however, there always exists an optimal solution that is a vertex of the feasible region.[2]

Examples

Example 1: Grain Nutrients

Suppose BU has hired you to optimize nutrition for campus dining. There are two possible grains they can offer, grain 1 and grain 2, and each contains the macronutrients found in the table below, plus cost per kg for each of the grains.

Macros	Starch	Proteins	Vitamins	Cost ($/kg)
Grain 1	5	4	2	0.6
Grain 2	7	2	1	0.35

The nutrition requirement per day of starch, proteins, and vitamins is 8, 15, and 3 respectively. Determine how much of each grain to buy such that BU spends as little but meets its nutrition requirements.

[1] A finite intersection of halfspaces is also called a "polyhedron;" in the common special case where the feasible region is bounded, it is called a "polytope."

[2] There are some annoying edge cases for unbounded feasible regions, for example the linear program $\max(x_1 + x_2)$ subject to $x_1 + x_2 = 1$.

Decision variables: amount of grain 1 (y_1) and grain 2 (y_2).

Objective: Minimize cost.

$$\min 0.6y_1 + 0.35y_2$$

Constraints:

$$\begin{aligned} 5y_1 + 7y_2 &\geq 8 && \text{(starch)} \\ 4y_1 + 2y_2 &\geq 15 && \text{(proteins)} \\ 2y_1 + 1y_2 &\geq 3 && \text{(vitamins)} \\ y_1, y_2 &\geq 0 && \text{(non-negativity)} \end{aligned}$$

Example 2: Transportation

You're working for a company that's producing widgets among two different factories and selling them from three different centers. Each month, widgets need to be transported from the factories to the centers. Below are the transportation costs from each factory to each center, along with the monthly supply and demand for each factory and center respectively. Determine how to route the widgets in a way that minimizes transportation costs.

Transit Cost	Center 1	Center 2	Center 3
Factory 1	5	5	3
Factory 2	6	4	1

- The supply per factory is 6 and 9 respectively.
- The demand per center is 8, 5, and 2 respectively.

Decision variables: x_{ij} is the number of widgets transported from factory i to center j.

Objective: Minimize cost.

$$\min \quad 5x_{11} + 5x_{12} + 3x_{13} + 6x_{21} + 4x_{22} + 1x_{23}$$

Constraints:

$$\begin{aligned} x_{11} + x_{12} + x_{13} &= 6 && \text{(Factor 1 supply)} \\ x_{21} + x_{22} + x_{23} &= 9 && \text{(Factor 2 supply)} \\ x_{11} + x_{21} &= 8 && \text{(Center 1 demand)} \\ x_{12} + x_{22} &= 5 && \text{(Center 2 demand)} \\ x_{13} + x_{23} &= 2 && \text{(Center 3 demand)} \\ x_{ij} &\geq 0 && \text{(non-negativity)} \end{aligned}$$

Converting to Normal Form

The "Normal Form" of a Linear Program looks like:

$$\begin{aligned} \max \quad & \mathbf{c}^T\mathbf{x} \\ \text{subject to} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \end{aligned}$$

Our Transportation problem had the LP:

$$\begin{aligned} \min \quad & 5x_{11} + 5x_{12} + 3x_{13} + 6x_{21} + 4x_{22} + 1x_{23} & \\ \text{subject to} \quad & x_{11} + x_{12} + x_{13} = 6 & \text{(Factor 1 supply)} \\ & x_{21} + x_{22} + x_{23} = 9 & \text{(Factor 2 supply)} \\ & x_{11} + x_{21} = 8 & \text{(Center 1 demand)} \\ & x_{12} + x_{22} = 5 & \text{(Center 2 demand)} \\ & x_{13} + x_{23} = 2 & \text{(Center 3 demand)} \\ & x_{ij} \geq 0 & \text{(non-negativity)} \end{aligned}$$

How can we convert it to normal form—a maximization problem with all less-than-or-equal-to constraints?

First observe that $x_{11} + x_{12} + x_{13} = 6$ is equivalent to having both inequalities

$$x_{11} + x_{12} + x_{13} \leq 6 \qquad \text{and} \qquad x_{11} + x_{12} + x_{13} \geq 6.$$

But, we need both to be $\leq$ inequalities! We transform them to

$$x_{11} + x_{12} + x_{13} \leq 6 \qquad \text{and} \qquad -x_{11} - x_{12} - x_{13} \leq -6.$$

The resulting LP in normal form is:

$$\begin{aligned} \max \quad & -5x_{11} - 5x_{12} - 3x_{13} - 6x_{21} - 4x_{22} - 1x_{23} & \\ \text{subject to} \quad & x_{11} + x_{12} + x_{13} \leq 6 & \text{(Factor 1 supply)} \\ & -x_{11} - x_{12} - x_{13} \leq -6 & \text{(Factor 1 supply)} \\ & x_{21} + x_{22} + x_{23} \leq 9 & \text{(Factor 2 supply)} \\ & -x_{21} - x_{22} - x_{23} \leq -9 & \text{(Factor 2 supply)} \\ & x_{11} + x_{21} \leq 8 & \text{(Center 1 demand)} \\ & -x_{11} - x_{21} \leq -8 & \text{(Center 1 demand)} \\ & x_{12} + x_{22} \leq 5 & \text{(Center 2 demand)} \\ & -x_{12} - x_{22} \leq -5 & \text{(Center 2 demand)} \\ & x_{13} + x_{23} \leq 2 & \text{(Center 3 demand)} \\ & -x_{13} - x_{23} \leq -2 & \text{(Center 3 demand)} \\ & x_{ij} \geq 0 & \text{(non-negativity)} \end{aligned}$$

Writing Problems We Know as Linear Programs

Independent Set

Given a graph $G = (V, E)$, each vertex i has weight w_i, find a maximum weighted *independent set*. S is an independent set if it does not contain both i and j for $(i, j) \in E$.

a. *Decision variables:* What are we try to solve for? A set of vertices S that is independent set: our variables are x_i for each vertex i, where we want $x_i = 1$ if i is in our independent set.

$$x_i = \begin{cases} 1 & i \in S \\ 0 & \text{o.w.} \end{cases}$$

b. *Constraints:* We cannot put vertices both i and j into the independent set if they share and edge, so

$$x_i + x_j \leq 1 \quad \forall\, (i, j) \in E.$$

and we can take at most one of each vertex and no negative quantities, so

$$x_i \in [0, 1] \quad \forall\, i \in V.$$

c. *Objective function:* We want to maximize the size of our independent set:

$$\max \sum_{i \in V} w_i x_i.$$

Note that this is a linear function.

We can put this all together, but that would give us an *integer program*, not a *linear program*. Asking that $x_i \in \{0, 1\}$ for all i is not a linear constraint. Solving this would precisely solve independent set, which we know to be NP-Hard. Instead, we relax this constraint to a fractional constraint so that it's linear, and just ask instead that $x_i \in [0, 1]$. This larger feasible region will still include all of the integer points, but will also include new fractional points, which can only have a better objective function. We call this the linear programming *relaxation*.

$$\begin{aligned} \max \sum_{i \in V} w_i x_i & \\ \text{s.t.} \quad x_i + x_j \leq 1 & \qquad (i, j) \in E \\ x_i \in [0, 1] & \qquad i \in V. \end{aligned}$$